```
import cv2
import numpy as np
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import time
from scipy.ndimage import gaussian_filter
```

```
# ============================
# OPC PARAMETERS
# ============================
num_iters = 100
lr = 0.005
threshold = 0.5
nx = 512                 # Simulation resolution
nm_per_pixel = 1.0       # Physical size per pixel (nm)
NA = 1.35                # Numerical aperture
wavelength = 193.0       # nm (ArF laser)
sharpness = 100
sigma = 2.0  # controls blurring, related to NA & wavelength

device = "cuda" if torch.cuda.is_available() else "cpu"
```

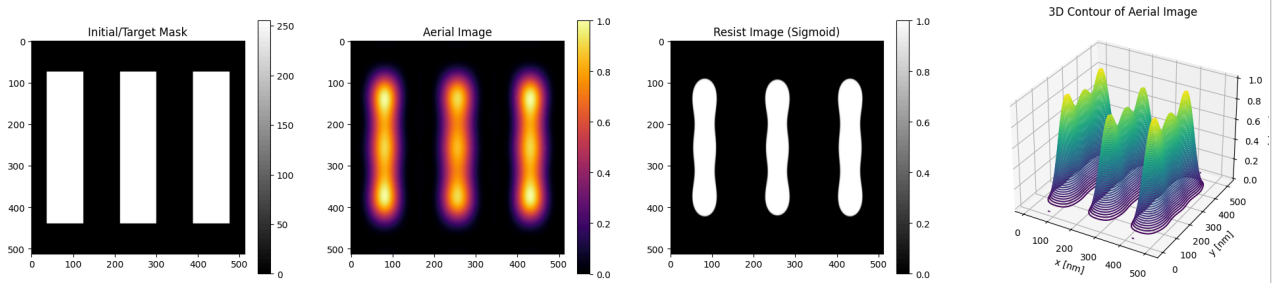
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# === Load and process target mask image ===
image_path = '/content/ILT_line_space_120.png'
mask_target_np = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
mask_target=torch.tensor(mask_target_np, dtype=torch.float32, device=device)
H, W = mask_target_np.shape
fov_X = W * nm_per_pixel
fov_Y = H * nm_per_pixel
fov = max(fov_X, fov_Y)
siz = max(H, W)
pad_H = (siz - H) // 2
pad_W = (siz - W) // 2
padded_img = cv2.copyMakeBorder(
    mask_target_np, pad_H, siz - H - pad_H, pad_W, siz - W - pad_W,
    borderType=cv2.BORDER_CONSTANT, value=0
)
resized_img = cv2.resize(padded_img, (nx, nx), interpolation=cv2.INTER_NEAREST)
resized_img_tensor = torch.tensor(resized_img, dtype=torch.float32, device=device)
resized_img_tensor = resized_img_tensor / 255.0
```

```
def compute_aerial_image_torch(weighted_result, NA, wavelength, nm_per_pixel):
    # Ensure input is a tensor and on the correct device, preserving requires_grad
    if not isinstance(weighted_result, torch.Tensor):
        mask = torch.tensor(weighted_result, dtype=torch.float32, device=device)
    else:
        mask = weighted_result.to(device)

    # Normalize mask if needed, but keep the require_grad property
    if mask.max() > 0:
        mask = mask / mask.max()
    else:
        mask = mask # Avoid division by zero

    # Reuse fov, nx, device from global scope
    dx = dy = fov / nx
    fx = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dx, device=device)) # Ensure on device
    fy = torch.fft.fftshift(torch.fft.fftfreq(nx, d=dy, device=device)) # Ensure on device
    FY, FX = torch.meshgrid(fy, fx, indexing='ij')
    FY = FY.to(device) # Ensure on device
    FX = FX.to(device) # Ensure on device

    # Ensure mask is complex for fft
    mask_complex = mask.to(torch.complex64)

    mask_ft = torch.fft.fftshift(torch.fft.fft2(mask_complex))
    cutoff_freq = NA / wavelength
    pupil_filter = ((FX**2 + FY**2).sqrt() <= cutoff_freq).float()
    pupil_filter = pupil_filter.to(device) # Ensure on device
    filtered_ft = mask_ft * pupil_filter
    ifft_img = torch.fft.ifft2(torch.fft.ifftshift(filtered_ft))
    aerial_image = torch.abs(ifft_img) ** 2
```

```
    # Normalize aerial image, but maintain require_grad
    if aerial_image.max() > 0:
        aerial_image = aerial_image / aerial_image.max()
    else:
        aerial_image = aerial_image # Avoid division by zero


    # Resist development using PyTorch operations
    resist_sigmoid = 1 / (1 + torch.exp(-sharpness * (aerial_image - threshold)))
    # resist_sigmoid = 0.5 * (1 + torch.cos(aerial_image))
    # resist_sigmoid = torch.clamp(resist_sigmoid, 0.0, 1.0)

    return resist_sigmoid, aerial_image # Return torch tensors
```

```
resized_img_tensor_for_aerial = torch.tensor(resized_img, dtype=torch.float32, device=device)

resist_image_sigmoid, resized_target=compute_aerial_image_torch(resized_img_tensor_for_aerial, NA, wavelength, nm_per_pixel
aerial_tensor = torch.tensor(resist_image_sigmoid, dtype=torch.float32)



x_coords = np.linspace(0, nx * nm_per_pixel, nx)
y_coords = np.linspace(0, nx * nm_per_pixel, nx)
X, Y = np.meshgrid(x_coords, y_coords)


# Visualization
plt.figure(figsize=(24,5))
plt.subplot(1,4,1)
plt.imshow(resized_img, cmap='gray')
plt.title('Initial/Target Mask')
plt.colorbar()

plt.subplot(1,4,2)
plt.imshow(resized_target, cmap='inferno')
plt.title('Aerial Image')
plt.colorbar()

plt.subplot(1,4,3)
plt.imshow(aerial_tensor, cmap='gray')
plt.title('Resist Image (Sigmoid)')
plt.colorbar()

# 4. 3D Contour of Aerial Image
ax = plt.subplot(1, 4, 4, projection='3d')
contour = ax.contour3D(X, Y, resized_target, levels=50, cmap='viridis')
ax.set_title('3D Contour of Aerial Image')
ax.set_xlabel('x [nm]')
ax.set_ylabel('y [nm]')
ax.set_zlabel('Intensity')
plt.show()

plt.tight_layout()
plt.show()
```

```
/tmp/ipython-input-1275710401.py:4: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.deta
  aerial_tensor = torch.tensor(resist_image_sigmoid, dtype=torch.float32)
```




```
<Figure size 640x480 with 0 Axes>
```

```
# # ============================
# # MODEL-BASED OPC FUNCTION
# # ============================
# def model_based_opc(target_layout: torch.Tensor):
#     """
#     Physics-driven model-based OPC
#     Input  : target_layout (binary tensor [0,1])
#     Output : corrected binary mask
```

```
#     """

#     target = target_layout.clone().float().to(device)

#     # --- initialize mask same as target ---
#     theta = torch.nn.Parameter(target.clone())

#     optimizer = torch.optim.Adam([theta], lr=lr)
#     loss_history = []

#     for it in range(num_iters):
#         optimizer.zero_grad()

#         # --- mask parameterization ---
#         mask = torch.sigmoid(theta)  # continuous mask [0,1]

#         # --- forward lithography model ---
#         resist, _ = compute_aerial_image_torch(
#             mask, NA, wavelength, nm_per_pixel
#         )

#         # --- OPC loss (edge / CD driven) ---
#         # MSE enforces CD matching
#         loss_cd = F.mse_loss(resist, target)

#         # TV regularization → Manhattan edges
#         tv_x = torch.mean(torch.abs(mask[:, :, :, 1:] - mask[:, :, :, :-1]))
#         tv_y = torch.mean(torch.abs(mask[:, :, 1:, :] - mask[:, :, :-1, :]))
#         tv_loss = tv_x + tv_y

#         loss = loss_cd + 0.01 * tv_loss
#         loss.backward()
#         optimizer.step()

#         loss_history.append(loss.item())

#         if it % 20 == 0:
#             print(f"[OPC] Iter {it:03d} | Loss={loss.item():.6f}")

#     # ============================
#     # FINAL BINARY OPC MASK
#     # ============================
#     final_mask = torch.sigmoid(theta.detach())
#     binary_mask = (final_mask > threshold).float()

#     return binary_mask.cpu().numpy(), loss_history
```

Below is diffrernt result

```
import torch
import torch.nn.functional as F

# # ============================
# # MODEL-BASED OPC FUNCTION
# # ============================
# def model_based_opc(target_layout: torch.Tensor):
#     """
#     Physics-driven Model-Based OPC

#     Returns:
#         binary_final_mask (numpy)
#         loss_history (list)
#         aerial_initial (numpy)
#         resist_final (numpy)
#         aerial_final (numpy)
#     """

#     target = target_layout.clone().float().to(device)

#     # Initialize mask parameters from target
#     theta = torch.nn.Parameter(target.clone())
#     optimizer = torch.optim.Adam([theta], lr=lr)

#     loss_history = []

#     # ----------------------------
#     # INITIAL AERIAL IMAGE (Before OPC)
#     # ----------------------------
#     with torch.no_grad():
#         initial_mask = torch.sigmoid(theta)
#         _, aerial_initial = compute_aerial_image_torch(
```

```
#             initial_mask, NA, wavelength, nm_per_pixel
#         )

#     # ----------------------------
#     # OPTIMIZATION LOOP
#     # ----------------------------
#     for it in range(num_iters):
#         optimizer.zero_grad()

#         # Continuous mask during optimization
#         mask = torch.sigmoid(theta)

#         # Lithography simulation
#         resist_sim, aerial_sim = compute_aerial_image_torch(
#             mask, NA, wavelength, nm_per_pixel
#         )

#         # --- OPC Objective: Match printed resist to target layout ---
#         loss_cd = F.mse_loss(resist_sim, target)

#         # --- Total Variation Regularization (smooth mask edges) ---
#         if mask.dim() == 2:
#             tv_x = torch.mean(torch.abs(mask[:, 1:] - mask[:, :-1]))
#             tv_y = torch.mean(torch.abs(mask[1:, :] - mask[:-1, :]))
#         elif mask.dim() == 4:
#             tv_x = torch.mean(torch.abs(mask[:, :, :, 1:] - mask[:, :, :, :-1]))
#             tv_y = torch.mean(torch.abs(mask[:, :, 1:, :] - mask[:, :, :-1, :]))
#         else:
#             raise ValueError("Unsupported mask dimension for TV loss")

#         tv_loss = tv_x + tv_y

#         # Total Loss
#         loss = loss_cd + 0.01 * tv_loss
#         loss.backward()

#         # Gradient stability
#         torch.nn.utils.clip_grad_norm_([theta], max_norm=1.0)
#         optimizer.step()

#         loss_history.append(loss.item())

#         if it % 20 == 0 or it == num_iters - 1:
#             print(f"[Model OPC] Iter {it:03d} | Loss={loss.item():.6f}")

#     # ----------------------------
#     # FINAL MASK & SIMULATION
#     # ----------------------------
#     final_mask = torch.sigmoid(theta.detach())
#     binary_final_mask = (final_mask > 0.5).float()

#     resist_final, aerial_final = compute_aerial_image_torch(
#         binary_final_mask, NA, wavelength, nm_per_pixel
#     )

#     # ----------------------------
#     # RETURN OUTPUTS
#     # ----------------------------
#     return (
#         binary_final_mask.detach().cpu().numpy(),   # 1️⃣ Final OPC mask
#         loss_history,                               # 2️⃣ Loss curve
#         aerial_initial.detach().cpu().numpy(),      # 3️⃣ Initial aerial
#         resist_final.detach().cpu().numpy(),        # 4️⃣ Final resist
#         aerial_final.detach().cpu().numpy()         # 5️⃣ Final aerial
#     )


import numpy as np
import torch
from scipy.ndimage import distance_transform_edt, gaussian_filter
from scipy.spatial import cKDTree
from skimage import measure

# ================ ENHANCED OPC ENGINE HELPERS ================

def get_mask_contour_points(mask, spacing_px=10):
    contours = measure.find_contours(mask, 0.25)
    if not contours:
        return [], []
    all_points, all_normals = [], []

    for contour in contours:
```

```
        if len(contour) < 20:
            continue
        n_samples = max(10, len(contour) // spacing_px)
        indices = np.linspace(0, len(contour)-1, n_samples, dtype=int)
        sampled_points = contour[indices]

        for i, point in enumerate(sampled_points):
            if i == 0:
                prev_point = sampled_points[-1]
                next_point = sampled_points[1]
            elif i == len(sampled_points)-1:
                prev_point = sampled_points[-2]
                next_point = sampled_points[0]
            else:
                prev_point = sampled_points[i-1]
                next_point = sampled_points[i+1]

            tangent = next_point - prev_point
            norm = np.linalg.norm(tangent)
            if norm > 0:
                tangent = tangent / norm
            normal = np.array([-tangent[1], tangent[0]])

            test_point = point + normal * 3
            ty = int(np.clip(test_point[0], 0, mask.shape[0]-1))
            tx = int(np.clip(test_point[1], 0, mask.shape[1]-1))
            if mask[ty, tx] > 0.5:
                normal = -normal

            all_points.append(point)
            all_normals.append(normal)

    return np.array(all_points), np.array(all_normals)


def calculate_corrections(points, normals, resist_contours, target_contours, dx_nm):
    if not resist_contours or not target_contours:
        return np.zeros(len(points))

    resist_points = np.vstack(resist_contours)
    target_points = np.vstack(target_contours)

    tree_resist = cKDTree(resist_points)
    tree_target = cKDTree(target_points)

    corrections_nm = []
    for p, n in zip(points, normals):
        _, idx_r = tree_resist.query(p)
        resist_point = resist_points[idx_r]

        _, idx_t = tree_target.query(resist_point)
        target_point = target_points[idx_t]

        vec = target_point - resist_point
        correction = -np.dot(vec, n) * dx_nm
        correction *= (1.0 + 0.5 * np.tanh(abs(correction)/50.0))
        corrections_nm.append(correction)

    return np.array(corrections_nm)


def apply_corrections_to_mask(mask, points, normals, corrections_nm, dx_nm, alpha=0.3):
    if len(points) == 0:
        return mask.copy()

    mask_binary = mask > 0.5
    sdf = distance_transform_edt(~mask_binary) - distance_transform_edt(mask_binary)

    sorted_idx = np.argsort(np.abs(corrections_nm))[::-1]

    for idx in sorted_idx[:min(100, len(points))]:
        y, x = points[idx]
        correction_nm = corrections_nm[idx]

        correction_px = alpha * (correction_nm / dx_nm)
        correction_px = np.clip(correction_px, -4.0, 4.0)

        yy, xx = np.ogrid[:mask.shape[0], :mask.shape[1]]
        dist_sq = (yy - y)**2 + (xx - x)**2
        influence = np.exp(-dist_sq / (2 * 4.0**2))
        sdf += influence * correction_px
```

```
    new_mask = (sdf < 0).astype(float)
    new_mask = gaussian_filter(new_mask, sigma=0.8)
    return (new_mask > 0.5).astype(float)

# ================ MODEL-BASED OPC (CONTOUR VERSION) ================

def model_based_opc(target_layout: torch.Tensor):
    """
    Contour-driven Model-Based OPC
    """

    target = target_layout.detach().cpu().numpy().astype(float)
    mask = target.copy()

    loss_history = []

    # Initial aerial image
    with torch.no_grad():
        _, aerial_initial = compute_aerial_image_torch(
            torch.tensor(mask, device=device).float(),
            NA, wavelength, nm_per_pixel
        )

    for it in range(num_iters):
        resist_t, _ = compute_aerial_image_torch(
            torch.tensor(mask, device=device).float(),
            NA, wavelength, nm_per_pixel
        )
        resist = resist_t.detach().cpu().numpy()

        resist_contours = measure.find_contours(resist, 0.5)
        target_contours = measure.find_contours(target, 0.5)

        points, normals = get_mask_contour_points(mask)
        corrections_nm = calculate_corrections(points, normals, resist_contours, target_contours, dx_nm)

        mask = apply_corrections_to_mask(mask, points, normals, corrections_nm, dx_nm)

        loss = float(np.mean(corrections_nm**2)) if len(corrections_nm) > 0 else 0.0
        loss_history.append(loss)

        if it % 5 == 0 or it == num_iters - 1:
            print(f"[Contour OPC] Iter {it:03d} | EPE Loss = {loss:.4f}")

    final_mask_tensor = torch.tensor(mask, device=device).float()
    resist_final, aerial_final = compute_aerial_image_torch(
        final_mask_tensor, NA, wavelength, nm_per_pixel
    )

    return (
        mask.astype(np.float32),                    # 1️⃣ Final OPC mask
        loss_history,                               # 2️⃣ Loss curve
        aerial_initial.detach().cpu().numpy(),      # 3️⃣ Initial aerial
        resist_final.detach().cpu().numpy(),        # 4️⃣ Final resist
        aerial_final.detach().cpu().numpy()         # 5️⃣ Final aerial
    )
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
# import torch
# import torch.nn.functional as F
# import numpy as np

# # ----------------------------
# # 1️⃣ Gaussian PSF (Imaging Model)
# # ----------------------------
# def gaussian_psf(size=41, sigma=2.0, device="cpu"):
#     ax = torch.arange(-size//2 + 1., size//2 + 1., device=device)
#     xx, yy = torch.meshgrid(ax, ax, indexing='ij')
#     kernel = torch.exp(-(xx**2 + yy**2) / (2 * sigma**2))
#     kernel = kernel / kernel.sum()
#     return kernel.unsqueeze(0).unsqueeze(0)  # shape (1,1,H,W)

# # ----------------------------
# # 2️⃣ Forward Lithography Model
# # ----------------------------
# def lithography_forward(mask, psf, Ith=0.5, alpha=0.03):
#     aerial = F.conv2d(mask, psf, padding="same")
#     resist = torch.sigmoid((aerial - Ith) / alpha)
#     return aerial, resist
```

```
# # ----------------------------
# # 3️⃣ Total Variation Regularization
# # ----------------------------
# def total_variation(mask):
#     dx = torch.abs(mask[:, :, :, :-1] - mask[:, :, :, 1:]).mean()
#     dy = torch.abs(mask[:, :, :-1, :] - mask[:, :, 1:, :]).mean()
#     return dx + dy

# # ----------------------------
# # 4️⃣ Model-Based OPC Optimization
# # ----------------------------
# def model_based_opc(
#     target_pattern,
#     iterations=150,
#     lr=0.005,
#     tv_weight=0.01,
#     device=device
# ):
#     """
#     target_pattern : tensor or numpy array (H, W) binary wafer target
#     """
#     if isinstance(target_pattern, torch.Tensor):
#         target = target_pattern.clone().detach().to(device)
#     else:
#         target = torch.tensor(target_pattern, dtype=torch.float32, device=device)

#     if target.dim() == 2:
#         target = target.unsqueeze(0).unsqueeze(0)  # (1,1,H,W)

#     psf = gaussian_psf(device=device)

#     # Initial mask = target
#     mask = target.clone().detach().requires_grad_(True)
#     optimizer = torch.optim.Adam([mask], lr=lr)
#     loss_history = []

#     # Initial aerial before OPC
#     with torch.no_grad():
#         aerial_initial, _ = lithography_forward(mask, psf)

#     print(f"Starting MB-OPC Optimization for {iterations} iterations...")

#     # ---------------- Iterative MB-OPC Loop ----------------
#     for i in range(iterations):
#         optimizer.zero_grad()

#         aerial, resist = lithography_forward(mask, psf)

#         loss_l2 = F.mse_loss(resist, target)
#         loss_tv = tv_weight * total_variation(mask)
#         loss = loss_l2 + loss_tv

#         loss.backward()
#         optimizer.step()

#         # Keep mask in valid range
#         with torch.no_grad():
#             mask.clamp_(0.0, 1.0)

#         loss_history.append(loss.item())

#         if i % 20 == 0 or i == iterations - 1:
#             print(f"  Iteration {i:03d}: Loss = {loss.item():.6f}")

#     # ---------------- Final Outputs ----------------
#     with torch.no_grad():
#         aerial_final, resist_final = lithography_forward(mask, psf)
#         binary_final_mask = (mask > 0.5).float()

#     return (
#         binary_final_mask.detach().cpu().numpy(),  # 1️⃣ Final OPC mask
#         loss_history,                              # 2️⃣ Loss curve
#         aerial_initial.detach().cpu().numpy(),     # 3️⃣ Initial aerial
#         resist_final.detach().cpu().numpy(),       # 4️⃣ Final resist
#         aerial_final.detach().cpu().numpy()        # 5️⃣ Final aerial
#     )
```

```
# import torch
# import torch.nn.functional as F
# import numpy as np

# # ----------------------------
```

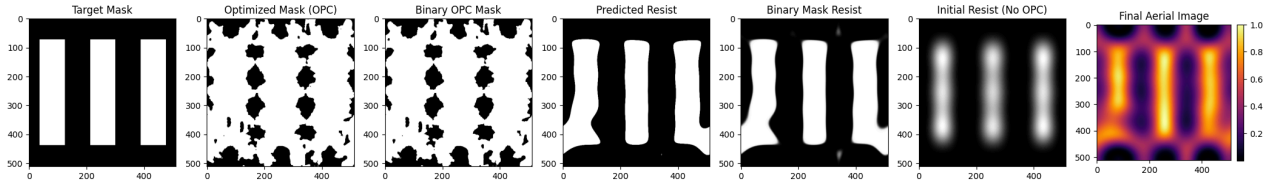
```
# # Model-Based OPC Optimization (Fourier Version)
# # ----------------------------
# def model_based_opc(
#     target_pattern,
#     iterations=120,
#     lr=0.005,
#     tv_weight=0.001,
#     device=device
# ):
#     """
#     target_pattern : tensor or numpy array (H, W) binary wafer target
#     """
#     if isinstance(target_pattern, torch.Tensor):
#         target = target_pattern.clone().detach().to(device)
#     else:
#         target = torch.tensor(target_pattern, dtype=torch.float32, device=device)

#     # Ensure target is (H, W) for the Fourier model
#     if target.dim() == 4:
#         target = target.squeeze()
#     elif target.dim() == 3:
#         target = target.squeeze()

#     # We use a parameter 'theta' and a sigmoid to represent the mask [0, 1] differentiably
#     # This is more stable than clamping the mask directly
#     theta = torch.nn.Parameter(torch.logit(torch.clamp(target.clone(), 1e-6, 1-1e-6)))

#     optimizer = torch.optim.Adam([theta], lr=lr)
#     loss_history = []

#     # Capture initial state for comparison
#     with torch.no_grad():
#         initial_mask = torch.sigmoid(theta)
#         resist_initial, aerial_initial = compute_aerial_image_torch(initial_mask, NA, wavelength, nm_per_pixel)

#     print(f"Starting Fourier-Based MB-OPC Optimization for {iterations} iterations...")

#     for i in range(iterations):
#         optimizer.zero_grad()

#         # Generate continuous mask
#         mask = torch.sigmoid(theta)

#         # Use the actual lithography simulator inside the loop
#         resist, aerial = compute_aerial_image_torch(mask, NA, wavelength, nm_per_pixel)

#         # Loss: Mismatch between printed resist and target + TV Regularization
#         loss_mse = F.mse_loss(resist, target)

#         # Total Variation (TV) to encourage clean edges
#         loss_tv = tv_weight * (torch.abs(mask[:, 1:] - mask[:, :-1]).mean() +
#                                torch.abs(mask[1:, :] - mask[:-1, :]).mean())

#         loss = loss_mse + loss_tv
#         loss.backward()
#         optimizer.step()

#         loss_history.append(loss.item())

#         if i % 20 == 0 or i == iterations - 1:
#             print(f"  Iteration {i:03d}: Loss = {loss.item():.6f}")

#     # ---------------- Final Outputs ----------------
#     with torch.no_grad():
#         final_mask = torch.sigmoid(theta)
#         binary_final_mask = (final_mask > 0.5).float()

#         # Simulate final print from the binary mask
#         resist_final, aerial_final = compute_aerial_image_torch(binary_final_mask, NA, wavelength, nm_per_pixel)

#     return (
#         binary_final_mask.cpu().numpy(),    # 1   Final Binary OPC mask
#         loss_history,                       # 2   Loss curve
#         aerial_initial.cpu().numpy(),       # 3   Initial aerial
#         resist_final.cpu().numpy(),        # 4   Final resist
#         aerial_final.cpu().numpy()         # 5   Final aerial
#     )
```

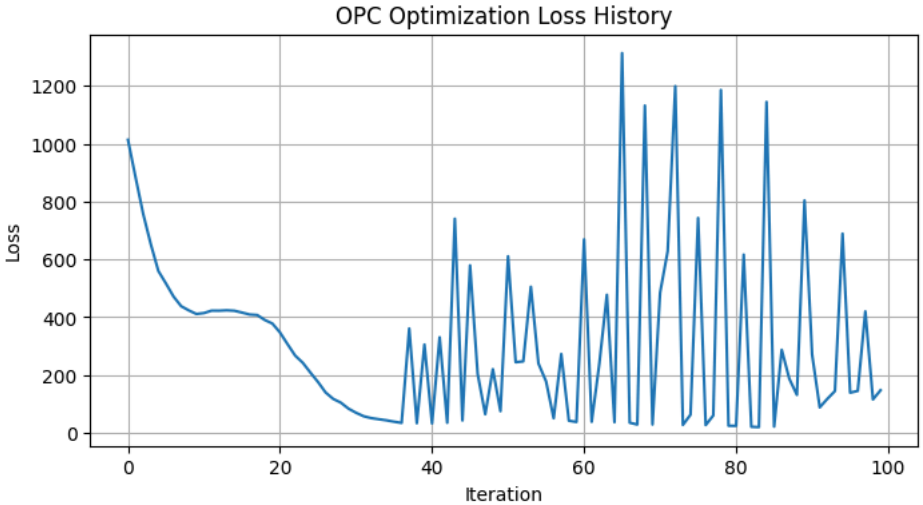
```
# # ============================
# # START TIMER
# # ============================
# start_time = time.time()
```

```
# # ============================
# # RUN MODEL-BASED OPC
# # ============================
# loss_history, resist_final_output, final_aerial_output = model_based_opc(resized_img_tensor,compute_aerial_image_torch, NA

# # Convert outputs to tensors for further simulation
# resist_final_tensor = torch.tensor(resist_final_output, dtype=torch.float32, device=device)
# aerial_final_tensor = torch.tensor(final_aerial_output, dtype=torch.float32, device=device)

# # ============================
# # DERIVE OPC MASK (Binary from resist thresholding)
# # ============================
# binary_opc_mask = (resist_final_output > 0.5).astype(np.float32)
# binary_opc_mask_tensor = torch.tensor(binary_opc_mask, dtype=torch.float32, device=device)

# # Simulate aerial + resist for binary OPC mask
# resist_binary_tensor, aerial_binary_tensor = compute_aerial_image_torch(
#     binary_opc_mask_tensor, NA, wavelength, nm_per_pixel
# )
# resist_binary_np = resist_binary_tensor.cpu().numpy()

# # ============================
# # PLOTTING
# # ============================
# fig, ax = plt.subplots(1, 7, figsize=(21, 5))

# # 1. Target layout
# ax[0].imshow(resized_img, cmap='gray')
# ax[0].set_title('Target Layout')

# # 2. Final resist from OPC optimization
# ax[1].imshow(resist_final_output.squeeze(), cmap='gray')
# ax[1].set_title('Resist (Model OPC)')

# # 3. Binary OPC mask
# ax[2].imshow(binary_opc_mask.squeeze(), cmap='gray')
# ax[2].set_title('Binary OPC Mask')

# # 4. Resist after binary mask simulation
# ax[3].imshow(resist_binary_np.squeeze(), cmap='gray')
# ax[3].set_title('Resist (Binary OPC Mask)')

# # 5. Initial resist (before OPC) for comparison
# with torch.no_grad():
#     init_resist_tensor, _ = compute_aerial_image_torch(
#         torch.tensor(resized_img, dtype=torch.float32, device=device),
#         NA, wavelength, nm_per_pixel
#     )
# ax[4].imshow(init_resist_tensor.cpu().numpy().squeeze(), cmap='gray')
# ax[4].set_title('Initial Resist (No OPC)')

# # 6. Final aerial image
# im = ax[5].imshow(final_aerial_output.squeeze(), cmap='inferno')
# ax[5].set_title('Final Aerial (Model OPC)')
# fig.colorbar(im, ax=ax[5], fraction=0.046, pad=0.04)

# # 7. Aerial from binary OPC mask
# ax[6].imshow(aerial_binary_tensor.cpu().numpy().squeeze(), cmap='inferno')
# ax[6].set_title('Aerial (Binary OPC Mask)')

# plt.tight_layout()
# plt.show()

# # ============================
# # LOSS CURVE
# # ============================
# plt.figure()
# plt.plot(loss_history)
# plt.title('OPC Optimization Loss')
# plt.xlabel('Iteration')
# plt.ylabel('Loss')
# plt.grid(True)
# plt.show()

# # ============================
# # END TIMER
# # ============================
# end_time = time.time()
# runtime = end_time - start_time
# print(f"Model-Based OPC runtime: {runtime:.2f} seconds")
```

```
# ============================
# START TIMER
# ============================
start_time = time.time()

# Define missing parameter dx_nm (nm per pixel in simulation grid)
dx_nm = fov / nx

# ============================
# RUN MODEL-BASED OPC
# ============================
# We use resized_img_tensor from cell ni22nK1ZFpKa
binary_opc_mask, loss_history, initial_aerial_output, resist_final_output, final_aerial_output = model_based_opc(
    resized_img_tensor)

# Optimized mask
optimized_mask = binary_opc_mask

# Threshold to ensure binary mask
binary_predicted_mask = (optimized_mask > 0.5).astype(np.float32)
binary_predicted_mask_tensor = torch.tensor(binary_predicted_mask, dtype=torch.float32, device=device)

# Simulate aerial + resist for the final binary predicted mask using the Fourier model
resist_binary_predicted_tensor, aerial_binary_predicted_tensor = compute_aerial_image_torch(
    binary_predicted_mask_tensor, NA, wavelength, nm_per_pixel
)
resist_binary_predicted_np = resist_binary_predicted_tensor.cpu().numpy()

# ============================
# PLOTTING RESULTS
# ============================
fig, ax = plt.subplots(1, 7, figsize=(21, 5))

ax[0].imshow(resized_img, cmap='gray')
ax[0].set_title('Target Mask')

ax[1].imshow(optimized_mask.squeeze(), cmap='gray')
ax[1].set_title('Optimized Mask (OPC)')

ax[2].imshow(binary_predicted_mask.squeeze(), cmap='gray')
ax[2].set_title('Binary OPC Mask')

# Re-simulating with Gaussian smoothing for visualization
with torch.no_grad():
    aerial_smooth = gaussian_filter(resist_final_output, sigma=sigma)
    aerial_tens = torch.tensor(aerial_smooth, dtype=torch.float32, device=device)
    resist_visual = torch.sigmoid(sharpness * (aerial_tens - threshold))

ax[3].imshow(resist_visual.cpu().numpy().squeeze(), cmap='gray')
ax[3].set_title('Predicted Resist')

ax[4].imshow(resist_binary_predicted_np.squeeze(), cmap='gray')
ax[4].set_title('Binary Mask Resist')

ax[5].imshow(resist_final_output.squeeze(), cmap='gray')
ax[5].set_title('Initial Resist (No OPC)')

im = ax[6].imshow(final_aerial_output.squeeze(), cmap='inferno')
ax[6].set_title('Final Aerial Image')
fig.colorbar(im, ax=ax[6], fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()

# Loss Curve
plt.figure(figsize=(8, 4))
plt.plot(loss_history)
plt.title('OPC Optimization Loss History')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.grid(True)
plt.show()

end_time = time.time()
print(f"Model-Based OPC runtime: {end_time - start_time:.2f} seconds")
```

```
[Contour OPC] Iter 000 | EPE Loss = 1013.5624
[Contour OPC] Iter 005 | EPE Loss = 516.5524
[Contour OPC] Iter 010 | EPE Loss = 414.4137
[Contour OPC] Iter 015 | EPE Loss = 416.1068
[Contour OPC] Iter 020 | EPE Loss = 347.1352
[Contour OPC] Iter 025 | EPE Loss = 175.9597
[Contour OPC] Iter 030 | EPE Loss = 69.6300
[Contour OPC] Iter 035 | EPE Loss = 38.6610
[Contour OPC] Iter 040 | EPE Loss = 32.6154
[Contour OPC] Iter 045 | EPE Loss = 579.3197
[Contour OPC] Iter 050 | EPE Loss = 610.9469
[Contour OPC] Iter 055 | EPE Loss = 178.2174
[Contour OPC] Iter 060 | EPE Loss = 669.1145
[Contour OPC] Iter 065 | EPE Loss = 1313.5024
[Contour OPC] Iter 070 | EPE Loss = 484.9540
[Contour OPC] Iter 075 | EPE Loss = 743.3387
[Contour OPC] Iter 080 | EPE Loss = 24.2091
[Contour OPC] Iter 085 | EPE Loss = 21.6537
[Contour OPC] Iter 090 | EPE Loss = 270.5601
[Contour OPC] Iter 095 | EPE Loss = 138.9148
[Contour OPC] Iter 099 | EPE Loss = 147.4378
```






```
Model-Based OPC runtime: 90.46 seconds
```

```
# def calculate_opc_accuracy(target_np, resist_np, threshold=0.25):
#     """
#     Evaluate OPC quality by comparing printed resist to target layout.
#     """

#     # Normalize resist if needed
#     if resist_np.max() > 1.0:
#         resist_norm = resist_np / 255.0
#     else:
#         resist_norm = resist_np

#     # Binarize
#     resist_bin = (resist_norm > threshold).astype(np.uint8)
#     target_bin = (target_np > 0).astype(np.uint8)

#     TP = np.sum((target_bin == 1) & (resist_bin == 1))
#     TN = np.sum((target_bin == 0) & (resist_bin == 0))
#     FP = np.sum((target_bin == 0) & (resist_bin == 1))
#     FN = np.sum((target_bin == 1) & (resist_bin == 0))

#     total = target_bin.size

#     accuracy = ((TP + TN) / total) * 100 if total > 0 else 0
#     precision = (TP / (TP + FP)) * 100 if (TP + FP) > 0 else 0
#     recall = (TP / (TP + FN)) * 100 if (TP + FN) > 0 else 0

#     # Mean absolute CD error inside printed regions
#     abs_diff = np.abs(resist_norm - target_bin)
#     cd_error = np.mean(abs_diff[target_bin == 1]) if np.sum(target_bin == 1) > 0 else 0

#     return {
#         "accuracy_%": accuracy,
```

```
#         "precision_%": precision,
#         "recall_%": recall,
#         "mean_cd_error": cd_error
#     }


def calculate_opc_accuracy(target_np, resist_np, threshold=0.5):
    """
    Accuracy metrics for Model-Based OPC
    Compares printed resist with target layout
    """

    # Normalize resist if needed
    if resist_np.max() > 1.0:
        resist_norm = resist_np / 255.0
    else:
        resist_norm = resist_np

    # Binarize
    resist_bin = (resist_norm > threshold).astype(np.uint8)
    target_bin = (target_np > 0).astype(np.uint8)

    TP = np.sum((target_bin == 1) & (resist_bin == 1))
    TN = np.sum((target_bin == 0) & (resist_bin == 0))
    FP = np.sum((target_bin == 0) & (resist_bin == 1))
    FN = np.sum((target_bin == 1) & (resist_bin == 0))

    total_pixels = target_bin.size

    accuracy = ((TP + TN) / total_pixels) * 100 if total_pixels > 0 else 0
    precision = (TP / (TP + FP)) * 100 if (TP + FP) > 0 else 0
    recall = (TP / (TP + FN)) * 100 if (TP + FN) > 0 else 0

    # CD-like delta value (intensity mismatch inside intended features)
    target_norm = target_bin.astype(np.float32)
    abs_diff = np.abs(resist_norm - target_norm)
    delta_value = np.mean(abs_diff[target_bin == 1]) if np.sum(target_bin == 1) > 0 else 0

    return {
        'accuracy': accuracy,
        'precision': precision,
        'delta_value': delta_value
    }
```

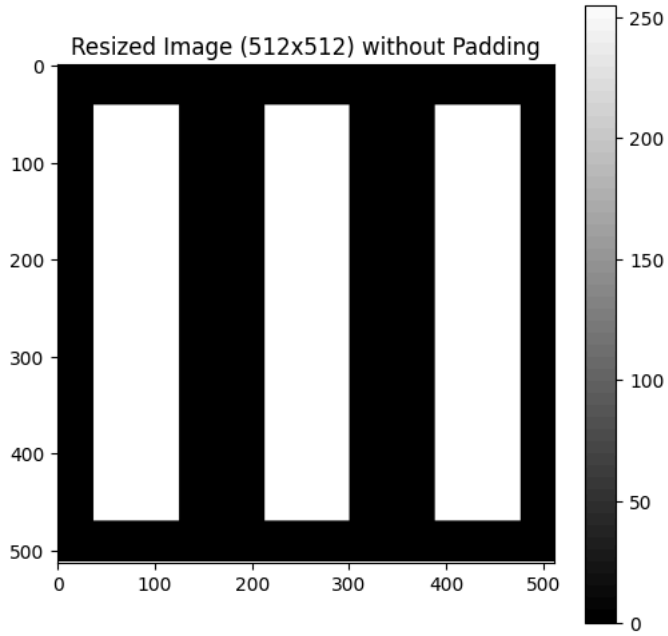
EPE

```
# import numpy as np
# import cv2
# from scipy.ndimage import distance_transform_edt

# def calculate_epe(target_np, resist_np, threshold=0.5, nm_per_pixel=1.0):
#     """
#     Compute Edge Placement Error (EPE) between target layout and printed resist.

#     Returns:
#         mean_epe_nm
#         max_epe_nm
#     """

#     # Normalize resist
#     if resist_np.max() > 1.0:
#         resist_norm = resist_np / 255.0
#     else:
#         resist_norm = resist_np

#     # Binarize
#     target_bin = (target_np > 0).astype(np.uint8)
#     resist_bin = (resist_norm > threshold).astype(np.uint8)

#     # Extract edges using Canny
#     target_edges = cv2.Canny((target_bin * 255).astype(np.uint8), 100, 200)
#     resist_edges = cv2.Canny((resist_bin * 255).astype(np.uint8), 100, 200)

#     # Distance transform of resist edges
#     resist_edge_inv = 1 - (resist_edges > 0).astype(np.uint8)
#     dist_map = distance_transform_edt(resist_edge_inv)

#     # Get coordinates of target edge pixels
#     edge_coords = np.column_stack(np.where(target_edges > 0))
```

```
#     if len(edge_coords) == 0:
#         return 0.0, 0.0

#     # Distances from each target edge to nearest resist edge
#     distances_pixels = dist_map[edge_coords[:, 0], edge_coords[:, 1]]
#     distances_nm = distances_pixels * nm_per_pixel

#     mean_epe = np.mean(distances_nm)
#     max_epe = np.max(distances_nm)

#     return mean_epe, max_epe
```

```
import cv2
import matplotlib.pyplot as plt

new_image_path = '/content/ILT_line_space_120.png'
new_mask_np = cv2.imread(new_image_path, cv2.IMREAD_GRAYSCALE)

if new_mask_np is None:
    print(f"Error: Could not load image from {new_image_path}")
else:
    print(f"Successfully loaded image with shape: {new_mask_np.shape}")

    # === Resizing to 512x512 without padding ===
    # Note: This will distort the aspect ratio if the original image is not square.
    resized_img_512 = cv2.resize(new_mask_np, (512, 512), interpolation=cv2.INTER_NEAREST)

    print(f"Resized image shape (without padding): {resized_img_512.shape}")

    # Display the resized image
    plt.figure(figsize=(6, 6))
    plt.imshow(resized_img_512, cmap='gray')
    plt.title('Resized Image (512x512) without Padding')
    plt.colorbar()
    plt.show()
```

```
Successfully loaded image with shape: (600, 700)
Resized image shape (without padding): (512, 512)
```



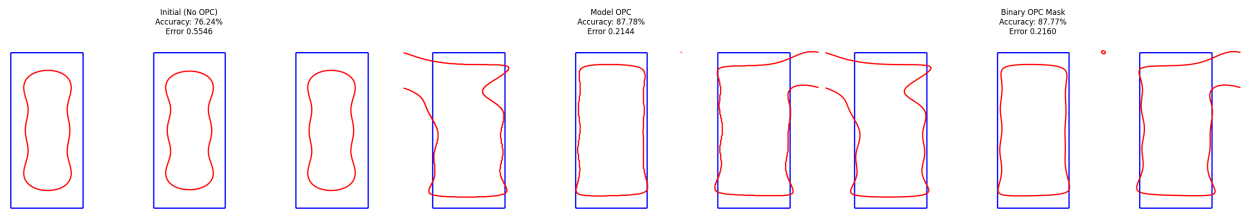

```
# metrics_initial_opc = calculate_opc_accuracy(resized_img_512, initial_aerial_output)
metrics_initial_opc = calculate_opc_accuracy(resized_img_512, aerial_tensor.squeeze().cpu().numpy())
# resist_tensor was renamed to resist_visual in the previous cell
metrics_opc = calculate_opc_accuracy(resized_img_512, resist_visual.squeeze().cpu().numpy())
metrics_binary_opc = calculate_opc_accuracy(resized_img_512, resist_binary_predicted_np.squeeze())

print("Initial OPC Metrics:", metrics_initial_opc)
print("Optimized OPC Metrics:", metrics_opc)
print("Binary OPC Mask Metrics:", metrics_binary_opc)
```

```
Initial OPC Metrics: {'accuracy': np.float64(76.24130249023438), 'precision': np.float64(100.0), 'delta_value': np.float32(0
Optimized OPC Metrics: {'accuracy': np.float64(87.78457641601562), 'precision': np.float64(91.73329451762822), 'delta_value'
Binary OPC Mask Metrics: {'accuracy': np.float64(87.77389526367188), 'precision': np.float64(91.70226254689821), 'delta_valu
```

EPE

```
# mean_epe_init, max_epe_init = calculate_epe(resized_img_512, initial_aerial_output, nm_per_pixel=nm_per_pixel)

# mean_epe_opc, max_epe_opc = calculate_epe(resized_img_512, resist_tensor.cpu().numpy(), nm_per_pixel=nm_per_pixel)

# mean_epe_binary, max_epe_binary = calculate_epe(resized_img_512, resist_binary_predicted_np, nm_per_pixel=nm_per_pixel)

# print(f"Initial EPE: Mean={mean_epe_init:.2f} nm | Max={max_epe_init:.2f} nm")
# print(f"Model OPC EPE: Mean={mean_epe_opc:.2f} nm | Max={max_epe_opc:.2f} nm")
# print(f"Binary OPC Mask EPE: Mean={mean_epe_binary:.2f} nm | Max={max_epe_binary:.2f} nm")
```

```
fig, ax = plt.subplots(1, 3, figsize=(28, 5))

# ----------------------------
# Initial (No OPC)
# ----------------------------
ax[0].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
# Using the initial aerial image converted to numpy
ax[0].contour(aerial_tensor.detach().cpu().numpy().squeeze(), levels=[0.5], colors='red', linewidths=2)
ax[0].set_title(
    f"Initial (No OPC)\n"
    f"Accuracy: {metrics_initial_opc['accuracy']:.2f}%\n"
    f"Error {metrics_initial_opc['delta_value']:.4f}"
)
ax[0].axis('off')

# ----------------------------
# Model-Based OPC Result
# ----------------------------
ax[1].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
# Fixed typo 'squeeeze' to 'squeeze' and added .cpu().numpy()
ax[1].contour(resist_visual.detach().cpu().numpy().squeeze(), levels=[0.5], colors='red', linewidths=2)
ax[1].set_title(
    f"Model OPC\n"
    f"Accuracy: {metrics_opc['accuracy']:.2f}%\n"
    f"Error {metrics_opc['delta_value']:.4f}"
)
ax[1].axis('off')

# ----------------------------
# Binary OPC Mask Result
# ----------------------------
ax[2].contour(resized_img_512, levels=[0.5], colors='blue', linewidths=2)
ax[2].contour(resist_binary_predicted_np.squeeze(), levels=[0.5], colors='red', linewidths=2)
ax[2].set_title(
    f"Binary OPC Mask\n"
    f"Accuracy: {metrics_binary_opc['accuracy']:.2f}%\n"
    f"Error {metrics_binary_opc['delta_value']:.4f}"
)
ax[2].axis('off')

plt.tight_layout()
plt.show()
```



Start coding or generate with AI.